Procura: demo script and function guide

What to click, what to say, and which function runs at every step

Harshit Chhabi | Operating Systems review | github.com/harshitchhabi/procura

This guide is for presenting the live demo. Sections 1 and 2 give the order of the demo with the exact steps. Section 3 explains every function in the project in the order the code runs, so any function a reviewer points at can be explained. Section 4 shows how to run the tokenizer, the parser and the other pieces on their own, and Section 5 lists what to do if something goes wrong. The review presentation has the same material as slides, with speaker notes.

Contents

1. Before the demo 2
2. The demo, step by step 3
3. Function reference, in the order the code runs 6
4. Run the pieces on their own 11
5. If something goes wrong in the demo 12

1. Before the demo

1.1 Start everything

Open two PowerShell windows. Port 8010 is used because another program on this machine occupies 8000.

```
# window 1: backend
cd D:\Downloads\Projects\OS\procura\backend
python seed_data.py              # resets the database and adds six demo tasks
uvicorn main:app --port 8010

# window 2: website
cd D:\Downloads\Projects\OS\procura\frontend
$env:VITE_API_URL="http://localhost:8010"
npm run dev
```

Open http://localhost:5173. The interactive API documentation is at http://localhost:8010/docs. A third window is used for the terminal dashboard and the commands in Demo 4. Run `python seed_data.py` again before each full run of the demo, because it resets the database; stop the backend first or restart it afterwards, since live resource allocations are held in memory.

1.2 The six demo tasks

Table 1. Tasks created by seed_data.py (priorities and sizes as seeded).

PID	Task	Priority	Burst (min)	Memory (MB)	CPU (%)
1	Finish OS term paper	High	180	512	30
2	Database systems project	High	240	768	45
3	Weekly grocery run	Low	45	32	5
4	Prepare capstone presentation slides	Medium	120	256	20
5	Render final project video	Medium	90	1600	70
6	Reply to internship emails	Medium	20	16	5

Resource pools: cpu 100 percent, memory 2048 MB, file 64 handles, database 20 connections, network 1000 Mbps.

1.3 Order and timing

Table 2. Demo order.

Demo	Topic	Minutes
1	Create a task	1
2	Compare the six schedulers	3
3	Memory contention and automatic recovery	2
4	A deadlock built and resolved, with the AI diagnosis	4
5	Ask the assistant	1
6	Terminal dashboard and benchmark	1

2. The demo, step by step

Demo 1: create a task

Table 3. Demo 1 steps.

Do	Say	What runs
Open **Processes**. Fill the form: name, description, priority High, burst 45, memory 256, CPU 20. Press **+ Create process**.	Every task is a process. It was admitted with the next PID and went NEW to READY at once.	`post_process` (main.py) calls `create_process`, which saves the row with `database.create_process` and calls `transition_state(pid, READY)`.
Point at the new row: PID 7, state READY.	The state machine is enforced on the server, not just in the interface.	`validate_transition` (states.py) checked NEW to READY.
Optional: on the API docs page, `PATCH /processes/7` with action `complete` while it is READY.	An illegal move is refused with HTTP 409.	`validate_transition` raises `InvalidTransitionError`; `patch_process` converts it to 409.

Demo 2: compare the six schedulers

Table 4. Demo 2 steps.

Do	Say	What runs
Open **Scheduler**, choose **FCFS**, run.	First come, first served. Look at the Gantt chart: each task runs to the end. A long task delays everything behind it.	`get_schedule` > `run_schedule('fcfs')` > `fcfs` > `_run_nonpreemptive` > `_build_metrics`. Only READY tasks are scheduled.
Choose **SJF**, run.	Shortest first among tasks that have arrived. Lowest average wait; the longest tasks wait most.	`sjf` with `_arrival_aware_order`.
Choose **Priority**, run.	Order by effective priority: user priority, deadline, waiting time, resource cost and fair share.	`priority_scheduling` with `effective_priority_score` and its five term functions.
Choose **Round Robin**, run.	Every task gets a four-minute turn in rotation. Response time is low; average wait is higher.	`round_robin(processes, quantum=4)`.
Choose **EEVDF**, run. Point at the lanes of the High-priority tasks.	Linux's scheduler since 6.6. High priority gets more slices through virtual time, and nothing starves.	`eevdf`, `nice_from_priority`, `weight_from_priority`.
Choose **MLFQ**, run.	Long tasks are demoted to lower queues; every 50 minutes everything is boosted to the top.	`mlfq`.
Point at the metrics panel on any run.	Waiting, turnaround and response time, utilisation, Jain's index, and slowdown and delay fairness.	`_build_metrics`, `jains_fairness_index`, `bounded_slowdown`.

Key sentence for the review: all six finish at the same time because none leaves the CPU idle; they differ only in who waits. SJF has the lowest average wait and a higher worst case; Round Robin and MLFQ respond fastest.

Demo 3: memory contention and recovery

Continue from Demos 1 and 2 without re-seeding, so the task created in Demo 1 (PID 7, High priority, 256 MB) exists. In the third window, give P3 and P1 their memory (the grocery task is the low-priority task that hogs it):

```
$A="http://localhost:8010"
Invoke-RestMethod -Method Post $A/resources/allocate -ContentType application/json -Body
'{"pid":3,"resource":"memory","amount":1200}'
Invoke-RestMethod -Method Post $A/resources/allocate -ContentType application/json -Body
'{"pid":1,"resource":"memory","amount":600}'
```

Table 5. Demo 3 steps.

Do	Say	What runs
Open **Dashboard**. Point at the memory bar: 1800 of 2048 MB used, 248 MB free.	Memory is nearly full. Several tasks cannot get what they need.	`allocate`; `utilization`.
Open **Error Monitor**. Press **Run detect now**.	One pass detects, explains and repairs. Four tasks cannot get their memory (P2, P4, P5 and P7) and each is handled differently. If you re-seeded and have no PID 7, expect three.	`detect` (main.py) > `run_detection` > `detect_memory_failure` for each task.
Point at the four rows: suspend_and_retry for P2, retry for P4 and P7, memory_failure_no_victim for P5.	P2 is High priority; the grocery task is strictly lower and its memory makes room, so it is suspended. P5 needs 1600 MB and nothing lower-priority can free that much, so nothing is evicted.	`recover_memory_failure`, `_lowest_priority_pid`, `run_recovery`.
Expand a diagnosis.	The explanation quotes the recorded numbers: 1600 MB requested, 248 MB free.	`diagnose` > `answer_question`, with incident facts.

Demo 4: a deadlock, built and resolved

This is the main demo. Restart from a fresh seed (`python seed_data.py`, then restart the backend). Use the third window.

Step 1: build the circular wait

```
$A="http://localhost:8010"; $J="application/json"
# 1. P1 takes all 64 file handles
Invoke-RestMethod -Method Post $A/resources/allocate -ContentType $J -Body
'{"pid":1,"resource":"file","amount":64}'
# 2. P3 takes all 20 database connections
Invoke-RestMethod -Method Post $A/resources/allocate -ContentType $J -Body
'{"pid":3,"resource":"database","amount":20}'
# 3. P1 asks for 5 database connections: fails, P1 now waits for P3
Invoke-RestMethod -Method Post $A/resources/allocate -ContentType $J -Body
'{"pid":1,"resource":"database","amount":5}'
# 4. P3 asks for 5 file handles: fails, P3 now waits for P1
Invoke-RestMethod -Method Post $A/resources/allocate -ContentType $J -Body
'{"pid":3,"resource":"file","amount":5}'
# the wait-for graph
Invoke-RestMethod $A/resources | ConvertTo-Json -Depth 5
```

Table 6. Step 1: what to say and what runs.

Call	Result to point at	What runs
P1 takes 64 file handles	success; file pool 100 percent	`allocate`

Call	Result to point at	What runs
P3 takes 20 database connections	success; database pool 100 percent	`allocate`
P1 asks for 5 connections	`success: false`, reason insufficient_capacity	`allocate` fails and calls `_record_wait`: edge P1 to P3
P3 asks for 5 file handles	`success: false`	`allocate` fails: edge P3 to P1
GET /resources	`waiting_for_graph: {1: [3], 3: [1]}`	`wait_graph`

Say: both pools are full and each task holds what the other needs. That is hold and wait, plus circular wait, with mutual exclusion and no preemption: all four deadlock conditions. On the Dashboard the file and database bars are at 100 percent; in `procura_top` the line WAIT-FOR GRAPH: 1 to [3], 3 to [1] appears in red.

Step 2: detect and resolve

Table 7. Step 2.

Do	Say	What runs
Open **Error Monitor**, press **Run detect now**.	One anomaly: a deadlock. Detection, explanation and repair all happen in this one call.	`detect` > `run_detection` > `detect_deadlock(resource_mgr, processes)` > `ResourceManager.detect_cycle` (depth-first search).
Point at the row: cycle 1 to 3 to 1, with each member's name and priority, what it holds and what it waits for.	The detector records who holds what, with names and priorities, so the explanation and the rule can be checked.	`detect_deadlock` builds the details text; `database.log_error` stores it.
Point at the recovery line: terminate_and_resume, target P3.	The rule is fixed: terminate the lowest-priority task in the cycle. P3 is the Low-priority grocery run.	`run_recovery` > `recover_deadlock` > `release_all_for_pid`; `database.log_recovery_action`.
Open **Dashboard**.	P3 is TERMINATED and its connections are free. P1 is READY again, still holding its file handles. The wait-for graph is empty.	`transition_state`; `utilization`.

Step 3: the AI explains it

Table 8. Step 3.

Do	Say	What runs
In Error Monitor, expand **Diagnosis** on the deadlock row.	The model was given the recorded incident, each member's priority, the fixed policy and passages on deadlocks. It explains the cycle, why P3 was chosen and what was released. It did not choose the action.	`diagnose` > `answer_question` > `search` (top 3 chunks from Deadlocks and Error_Recovery) > `_build_prompt` > `_call_groq`.
Open **AI Assistant**, select **P3**, ask: Why was this task terminated, and was that the right decision?	The answer cites the recorded cycle and the rule, and says the decision was consistent with the policy.	`ask` > `_incident_facts_for` (error details, action executed, rule applied, current resources) > `answer_question`.

Be ready to say: the first sentences of an answer are often a generic definition; the part to read is the incident section. The model explains in words; fixed scripts act, so a wrong explanation cannot cause a wrong

action. If the network or key is unavailable, the assistant returns the retrieved passages and the facts, labelled offline, with the reason.

Demo 5: ask the assistant

Table 9. Demo 5 steps.

Do	Say	What runs
On **AI Assistant**, select process 5 and ask why it hit a memory allocation failure and what the system did.	It quotes the recorded numbers (1600 MB requested, 248 MB free) and says nothing was evicted.	`ask`, `_incident_facts_for`, `answer_question`.
Clear the process, ask: How is EEVDF different from Round Robin?	A concept question answered from the knowledge base.	`search` over 63 chunks, `answer_question`.

Demo 6: terminal dashboard and benchmark

```
# window 3, from the backend folder
$env:PROCURA_API_URL="http://localhost:8010"
python -m cli.procura_top

# in another window
python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42
```

Table 10. Demo 6 steps.

Do	Say	What runs
Start `procura_top`. Repeat the deadlock calls from Demo 4 in another window.	The same data as a terminal monitor, refreshed every two seconds. The wait-for graph appears and clears.	`fetch_json`, `build_process_table`, `build_resource_panel`, `build_ai_feed`, `render`.
Run the benchmark.	All six policies on identical jobs: same makespan, 228.67, different waiting times. SJF 34.0 minutes against 61.6 for FCFS.	`generate_poisson_workload`, `run_all`, `summarize`, `print_table`.

3. Function reference, in the order the code runs

Paths are relative to the `backend/` folder. Function names in code font can be searched for directly.

os_sim/states.py

The legal lifecycle moves.

Table 11. os_sim/states.py.

Function	What it does
`State`	The six state names: NEW, READY, RUNNING, BLOCKED, COMPLETED, TERMINATED.
`validate_transition(current, target)`	Raises `InvalidTransitionError` unless the move is in the table: NEW to READY; READY to RUNNING or TERMINATED; RUNNING to READY, BLOCKED, COMPLETED or TERMINATED; BLOCKED to READY or TERMINATED.

os_sim/process_manager.py

Creates tasks and moves them between states. Every state change goes through one function, so the rule is enforced in one place.

Table 12. os_sim/process_manager.py.

Function	What it does
`create_process(...)`	Saves a new task (database.create_process), then moves it NEW to READY.
`get_process(pid) / list_processes(state)`	Read one task (raises `ProcessNotFoundError` if absent) or all, optionally filtered by state.
`transition_state(pid, target)`	Validates the move, sets `started_at` when first RUNNING and `ready_since` on entering READY, saves.
`start_process / pause_process / resume_process`	READY to RUNNING; RUNNING to BLOCKED; BLOCKED to READY.
`complete_process / terminate_process`	Move to the two final states.
`increment_error_count(pid)`	Adds one to the task's error count (used by recovery).
`delete_process(pid)`	Removes the task.

database.py

Standard-library `sqlite3`, no ORM. Four tables: processes, errors, diagnoses, recovery_actions.

Table 13. database.py.

Function	What it does
`init_db / reset_db`	Create the tables; drop and recreate them (seed script and tests).
`create_process / get_process / list_processes / update_process / delete_process`	CRUD for the processes table.
`log_error(pid, type, details)`	Records a detected failure and returns its id.
`log_diagnosis(error_id, text) / log_recovery_action(error_id, type, target_pid)`	Attach the explanation and the action to an error.
`get_error_log()`	Joins the three history tables into the list the Error Monitor and `/ask` use.
`get_connection / db_cursor / now_iso`	Connection helpers and the timestamp format.

os_sim/scheduler.py: effective priority

Five small functions, each testable alone, added together.

Table 14. os_sim/scheduler.py: effective priority.

Function	What it does
`user_priority_score(priority)`	High 90, Medium 50, Low 20; a number is clamped to 0 to 100.
`deadline_urgency_score(deadline, now)`	0 beyond 48 hours; 100 once passed; linear between.
`waiting_time_score(ready_since, now)`	2 points per minute in READY, capped at 100: the anti-starvation term.
`resource_requirement_score(memory, cpu)`	Up to 60 for memory (of 2048 MB) plus up to 40 for CPU.
`fair_share_score(proc, all_processes, now, half_life)`	Zero or negative: a penalty for using more than 1/n of the decayed past usage; `_decay_factor` halves usage every 60 minutes (Slurm's idea).

Function	What it does
`effective_priority_score(proc, now, all_processes)`	The sum. Used by `priority_scheduling` and by the recovery rules to compare tasks.

os_sim/scheduler.py: policies and metrics

All policies take the same list of tasks and return order, Gantt slices and metrics.

Table 15. os_sim/scheduler.py: policies and metrics.

Function	What it does
`fcfs(processes)`	Dispatch in arrival order.
`sjf(processes)`	Among tasks that have arrived, run the shortest burst.
`priority_scheduling(processes, now)`	Among tasks that have arrived, run the highest effective priority.
`_arrival_aware_order(processes, key)`	The shared rule behind SJF and Priority; idles only if nothing has arrived.
`_run_nonpreemptive(processes, order, algorithm)`	Turns an order into Gantt slices and metrics.
`round_robin(processes, quantum)`	FIFO ready queue; each turn at most the quantum (4 minutes); inner `admit` brings in arrivals.
`nice_from_priority / weight_from_priority`	High -5, Medium 0, Low +5 as a nice value; weight = 1024 / 1.25^nice.
`eevdf(processes, base_slice)`	Virtual runtime, lag, eligibility and earliest virtual deadline via a min-heap.
`mlfq(processes, num_queues, base_quantum, boost_interval)`	Three queues (4, 8, 16 minutes), cumulative allotment, boost every 50 minutes; inner `admit` and `maybe_boost`.
`run_schedule(algorithm, processes, ...)`	Dispatch by name to any of the six.
`_build_metrics(...)`	Per-task and average turnaround, waiting and response; utilisation; both fairness indices; slowdown.
`jains_fairness_index(values)`	(sum x)^2 / (n times sum x^2), between 1/n and 1.
`bounded_slowdown(turnaround, burst, bound)`	max(turnaround / max(burst, 1 minute), 1).
`_parse_arrival / _relative_arrivals`	Accept an ISO date or a number of minutes; shift arrivals so the earliest is 0.

os_sim/resource_manager.py

Five pools and the wait-for graph. The state is held in memory and resets with the server.

Table 16. os_sim/resource_manager.py.

Function	What it does
`describe_amount(resource, amount)`	Text with units: '64 file handles', '1200 MB of memory', '20 database connections'.
`ResourceManager.allocate(pid, resource, amount)`	Grants the request, or fails and records a wait-for edge to the holder; checks lock order first.
`release / release_all_for_pid`	Free one resource or everything a task holds, and clear its wait edges.
`available / utilization`	Free amount of a pool; use of all five pools.
`_record_wait(pid, resource)`	Adds the edge from the requester to each holder.
`wait_graph()`	The graph as {pid: [pids it waits for]}.
`detect_cycle()`	Depth-first search with three colours; returns the PIDs on a cycle, or nothing.

Function	What it does
`check_lock_order_violation(pid, resource)`	Logs when a task holding a later resource asks for an earlier one (order: cpu, memory, file, database, network).
`check_timeouts(now)`	Aborts any wait longer than 30 seconds.
`register_max_need / _would_be_safe / _held_resources`	Banker's-style avoidance (off by default; switched by `POST /resources/avoidance-mode`).

recovery/detector.py

One entry point, `run_detection`, runs every detector.

Table 17. recovery/detector.py.

Function	What it does
`run_detection(processes, resource_mgr, method)`	Deadlock check, then for each unfinished task a memory check, and for RUNNING tasks the anomaly check.
`detect_memory_failure(process, resource_mgr)`	Flags a task whose unmet memory requirement is larger than what is free (a task that already holds its memory is not flagged).
`detect_deadlock(resource_mgr, processes)`	Flags a cycle; the details name each member, its priority, what it holds and what it waits for.
`detect_burst_anomaly(process, running_minutes, resource_mgr, method)`	Compares the elapsed/burst ratio with the task's own last 30 ratios using one of the three tests below.
`ewma_flag / modified_zscore_flag / cusum_flag`	EWMA control limits; modified Z-score above 3.5; CUSUM above its threshold.
`reset_history / _record_ratio`	The rolling per-task history.

rag/ingestion.py: the tokenizer and the index

The tokenizer is the lexer-like part of the project.

Table 18. rag/ingestion.py: the tokenizer and the index.

Function	What it does
`tokenize_v2(text)`	Finds runs of letters and digits joined by '.', '-' or '_'; lowercases; indexes a compound whole and by its parts; drops 106 stopwords; stems. This is the default `tokenize`.
`tokenize_v1(text)`	The original, kept as the baseline for the evaluation (letters only, 49 stopwords, no stemming).
`_stem(word)`	Conservative suffix folding: sses, ies, ing, ed, es, s.
`_split_into_chunks / load_chunks`	Split each document on paragraph boundaries into chunks of at most about 600 characters: 63 chunks from 8 documents.
`TfidfIndex / build_index`	Weights each word by term share times idf, idf = ln((1+N)/(1+df))+1; `vectorize_query` turns a question into the same kind of vector.

rag/retrieval.py and rag/assistant.py

Search, then answer.

Table 19. rag/retrieval.py and rag/assistant.py.

Function	What it does
`search(query, top_k, source_filter, index)`	Cosine similarity against every chunk; returns the top 3, optionally limited to named documents.
`_cosine / get_index`	The similarity and the cached index.
`_format_process_context / _format_incident_facts / _build_prompt`	Assemble the four parts: instruction, process record, incident facts, passages.
`_call_groq / _call_anthropic / _call_openai`	One provider each; the first with a key is used (Groq first).
`_post_json(url, headers, body, attempts)`	Retries rate limits and server errors using the provider's own wait hint; returns the reason if it still fails.
`_extractive_fallback(...)`	Offline answer: passages and facts, with the reason the model did not answer.
`answer_question(question, process, top_k, source_filter, incident_facts)`	The whole pipeline; returns answer, mode, sources and fallback_reason.

recovery/diagnosis.py and recovery/recovery.py

The AI explains; fixed rules act.

Table 20. recovery/diagnosis.py and recovery/recovery.py.

Function	What it does
`diagnose(finding, process)`	Asks `answer_question` to explain the finding, passing the details and the fixed policy for that failure type as facts.
`run_recovery(finding, resource_mgr)`	Chooses the script by error type.
`recover_memory_failure`	Retry; else suspend the lowest-priority holder only if its priority is strictly lower and freeing its memory makes the allocation fit; else leave the task pending and evict nobody. Always increments the error count.
`recover_deadlock`	Terminate the lowest-priority task in the cycle, release all its resources, return the other blocked members to READY.
`recover_timeout / recover_crash`	Move the task back to READY and add one to its error count.
`_lowest_priority_pid(pids, exclude)`	Finds the eviction candidate by effective priority.

main.py (the 16 endpoints)

Every interface uses only these.

Table 21. main.py (the 16 endpoints).

Function	What it does
`GET /health`	Liveness check.
`GET, POST /processes`	List; create.
`GET, PATCH, DELETE /processes/{pid}`	Read; change state with an action (start, pause, resume, complete, terminate); delete (releases its resources first).
`GET, POST /schedule`	Run one policy over READY tasks.
`GET /benchmark`	Run all six on a generated workload.
`GET /resources`	Use of each pool and the wait-for graph.

Function	What it does
`POST /resources/allocate, /resources/release`	Allocate or release a resource for a task.
`POST /resources/avoidance-mode`	Switch Banker's-style avoidance.
`POST /detect`	Detect, diagnose, recover, and log, once.
`POST /ask`	Ask the assistant, optionally about one process (adds `_incident_facts_for`).
`GET /errors`	The joined history.

benchmark/

Workloads and metrics.

Table 22. benchmark/.

Function	What it does
`generate_poisson_workload(n, arrival_rate, burst_shape, burst_scale, seed)`	Poisson arrivals and Gamma burst times, seeded.
`generate_workload(kind, n, load, seed)`	Gamma, heavy-tailed or bursty workloads at a chosen offered load.
`parse_swf(path)`	The parser: skips comments and blanks, splits on whitespace, rejects lines with fewer than 4 fields, converts, pads to 18 fields.
`swf_to_processes(jobs)`	Maps SWF jobs to Procura tasks: arrival shifted to 0, burst from run time, memory from used memory.
`summarize(result, ...) / makespan / utilization / starvation_metrics`	The standard metric set for one run.
`run_all / print_table / main (run_benchmark.py)`	Run all six policies and print the comparison.
`study / one_run / mean_ci / t_critical (study.py)`	The 495-workload study: means with 95% confidence intervals and paired differences against FCFS.

cli/procura_top.py

The terminal dashboard.

Table 23. cli/procura_top.py.

Function	What it does
`fetch_json(path)`	Calls the API.
`build_process_table / build_resource_panel / build_ai_feed`	Three panels: processes by state, resource bars with the wait-for graph, and the recent diagnosis and recovery events.
`render(console) / main()`	Compose the layout and refresh every two seconds.

4. Run the pieces on their own

From the backend folder. None of these needs the server or the website.

```
# the tokenizer (lexer-like)
python -c "from rag.ingestion import tokenize; print(tokenize('Why did the Render-video task hit a
memory failure on kernel 6.6?'))"
# the search
python -c "from rag.retrieval import search; [print(round(r.score,3), r.chunk.source) for r in
```

```
search('How is EEVDF different from Round Robin?')]"
# the parser
python -c "from benchmark.workload import parse_swf; j =
parse_swf('benchmark/sample_traces/sample.swf'); print(len(j), j[0])"
# one scheduler by hand
python -c "from os_sim.scheduler import sjf;
print(sjf([{'pid':1,'name':'a','arrival_time':0,'burst_time':5},{'pid':2,'name':'b','arrival_time':1
,'burst_time':3}])['order'])"
# the benchmark, the study, the evaluations, the tests
python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42
python -m benchmark.study
python -m rag.eval_retrieval
python -m rag.eval_grounding              # needs a model key in backend/.env
python -m pytest tests -q                 # 101 tests
```

5. If something goes wrong in the demo

Table 24. Problems and what to do.

Symptom	Likely cause	What to do
The website shows no data	The frontend is pointing at the wrong port	Set `VITE_API_URL` to the port the backend uses and restart `npm run dev`.
Allocation calls say the pool is not empty	The previous run's allocations are still in memory	Restart the backend (uvicorn) after `python seed_data.py`.
The assistant says it is offline	No key set, or the model was rate limited	Say so: the answer shows the retrieved passages and the reason. Retry after a few seconds.
Detect finds nothing	The deadlock calls were not made, or detection already ran	Re-seed, restart the backend and repeat Step 1.
Port already in use	Another program holds 8000 or 5173	Use 8010 for the backend and set `VITE_API_URL`.